
Sistemas multiprocesador y su programación

Los sistemas con múltiples procesadores (o múltiples núcleos de procesamiento) permiten ejecutar múltiples tareas o procesos en paralelo. En general, existen dos conceptos importantes con respecto al paralelismo que deben abordarse: la concurrencia y el paralelismo, ilustrados en la Figura 2.1.

Un sistema **concurrente** utiliza un único recurso computacional que ejecuta dos o varias tareas, haciendo progreso parcial en cada una de forma multiplexada, sea por tiempo o por prioridad. Estos sistemas cuentan con un calendarizador a nivel de software que permite determinar qué tarea y por cuánto tiempo puede utilizar el procesador. Esto comúnmente se ven en cursos de Sistemas Operativos, que se encuentran fuera del alcance de este libro.

Un sistema **paralelo** ejecuta dos o varias tareas de forma simultánea repartiendo el trabajo entre dos o más recursos computacionales (o procesadores). A diferencia de un sistema concurrente, el sistema paralelo logra avance en paralelismo real, mientras que la concurrencia es un espejismo que da la impresión de paralelismo.

Asimismo, es necesario recordar el concepto de **hilo** (*thread*) y **proceso**. Ambos se refieren a fragmentos de algoritmo con instrucciones. Sin embargo, un proceso puede considerarse como un algoritmo en aislamiento, y puede corresponder a un programa entero. Un hilo, en cambio, puede formar parte de un proceso. En este caso, un proceso puede tener más de un hilo, permitiendo que ejecute múltiples tareas en paralelo. Por otra parte, los procesos tienen aislamiento en el espacio de direcciones virtual, mientras que los hilos de un mismo proceso comparten el espacio de direcciones. En términos más simples, un proceso no puede ver la memoria de otro proceso (inclusive, la misma dirección, puede tener dos datos completamente distintos), mientras que dos hilos de un mismo proceso, sí pueden. Esto se hace para garantizar integridad de los datos y evitar que un proceso pueda invadir y dañar a otro.

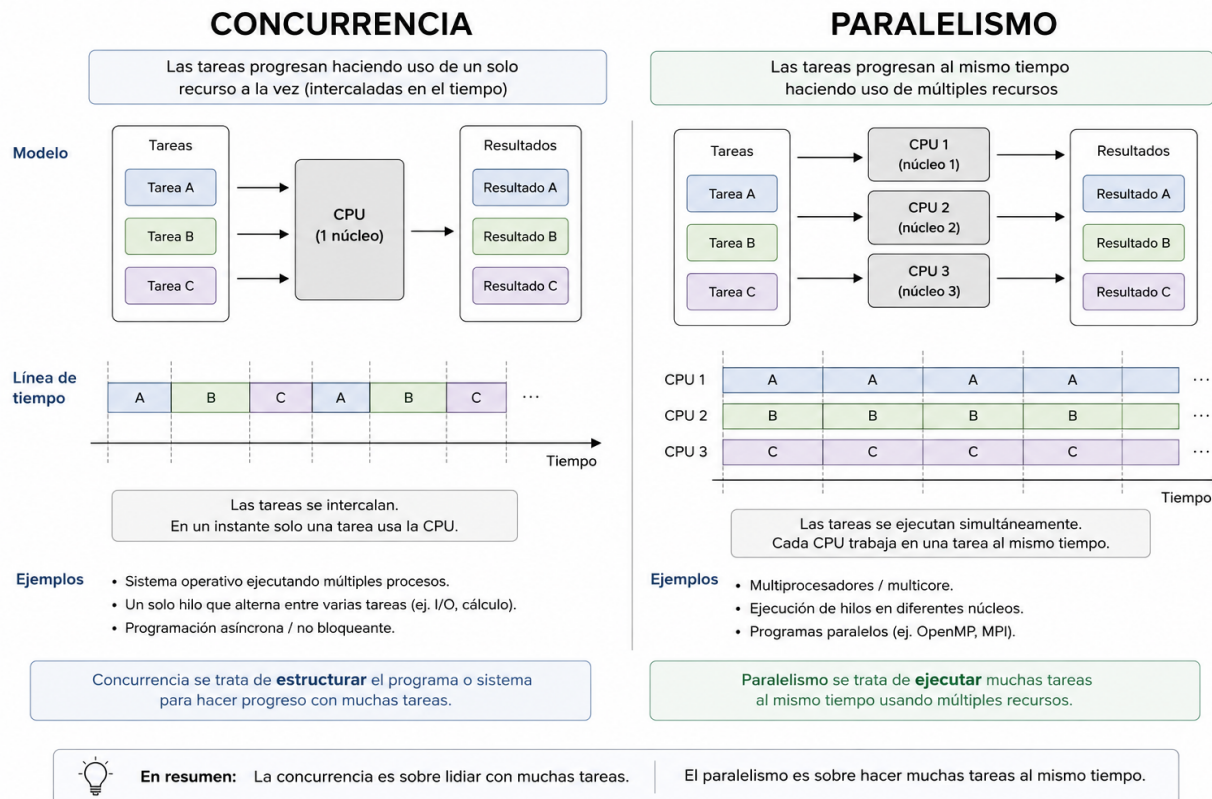


Figura 2.1: Diferencia entre paralelismo y concurrencia. Imagen generada con IA

Para lograr sistemas paralelos, es necesario ahondar en los sistemas multiprocesador, que son sistemas que cuentan con múltiples procesadores, que estén contenidos dentro del mismo *die* de silicio o en diferentes *sockets*. En el caso de que los múltiples procesadores se encuentren dentro del mismo chip de silicio, se refieren como núcleos, mientras que si se encuentran en distintos chips (o *sockets*), estos se llaman procesadores.

A lo largo de este capítulo se ahondará en la organización y programación de sistemas multiprocesadores.

2.1. Organización de Sistemas Multiprocesador

Como se introdujo con anterioridad, los microprocesadores (o unidades computacionales) pueden estar contenidos dentro del mismo *die* o en *sockets* distintos. Esto profundiza una diferencia significativa en cómo se interconectan e intercambian datos. En sistemas de servidores, es común encontrar híbridos que utilizan ambos paradigmas, mientras que en ordenadores personales, celulares o dispositivos empujados, es común encontrar sistemas multiprocesadores con núcleos en el mismo *die*.

Sistemas Multinúcleo

Los sistemas multinúcleo contienen múltiples unidades de procesamiento dentro del mismo *socket* o *die*. Una característica particular que tienen los sistemas multinúcleo es que todos sus núcleos de procesamiento se encuentran conectados a una memoria caché común L3, lo que maximiza el tiempo de comunicación entre las unidades de procesamiento.

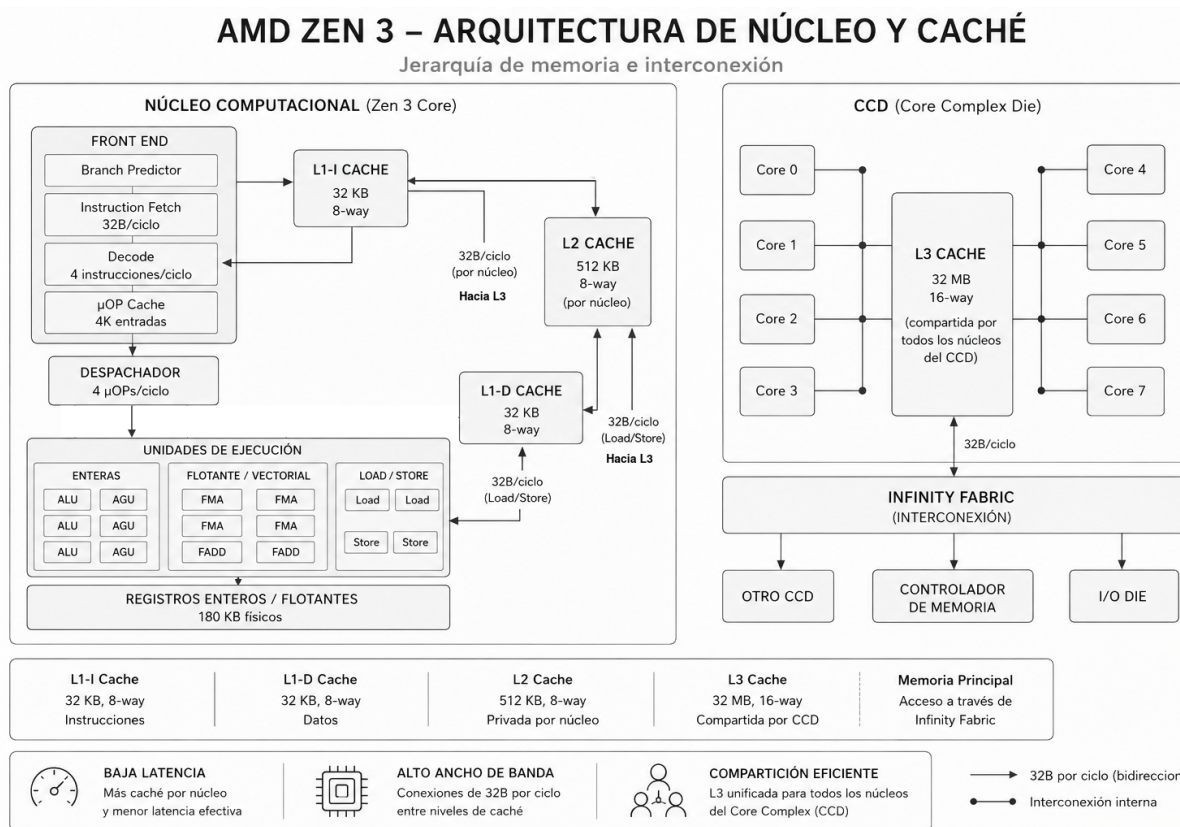


Figura 2.2: Organización del procesador AMD Zen 3. Los CCD son arreglos de núcleos. Los procesadores Ryzen contienen usualmente solo 1, mientras que los EPYC de servidores, tienen dos o más. Imagen generada con IA y editada por el autor.

La Figura 2.2 destaca esta particularidad, donde cada núcleo está equipado con sus propias caché L1 y L2, que completan la máquina computacional híbrida que se vio en el capítulo anterior. Adicional a eso, las caché L2 se comunican a la caché L3. Esto permite que cada núcleo intercambie datos con otros núcleos mediante escrituras/lecturas de memoria caché, explotando el sistema de coherencia de caché.

En esta microarquitectura, se encuentra el concepto de *Core Complex Die*, que forma grupos o *clusters* de núcleos computacionales, que pueden comunicarse con otros CCD. Esta configuración se encuentra mayormente en los procesadores de servidor de AMD (AMD EPYC). Cada CCD se comunica con otros CCD y con el controlador de memoria DDR mediante el bus de interconexión *Infinity Fabric*, como se ilustra en la Figura 2.3. Esto implica una serie de problemas, ya que la memoria L3 se fragmenta, generando el problema de fractura del espacio de direcciones. Sin embargo, la memoria RAM actúa como el entre unificador, por lo que se mitiga parcialmente. Los conceptos detrás del funcionamiento de comunicación extra CCD serán tratados como un sistema multisocket.

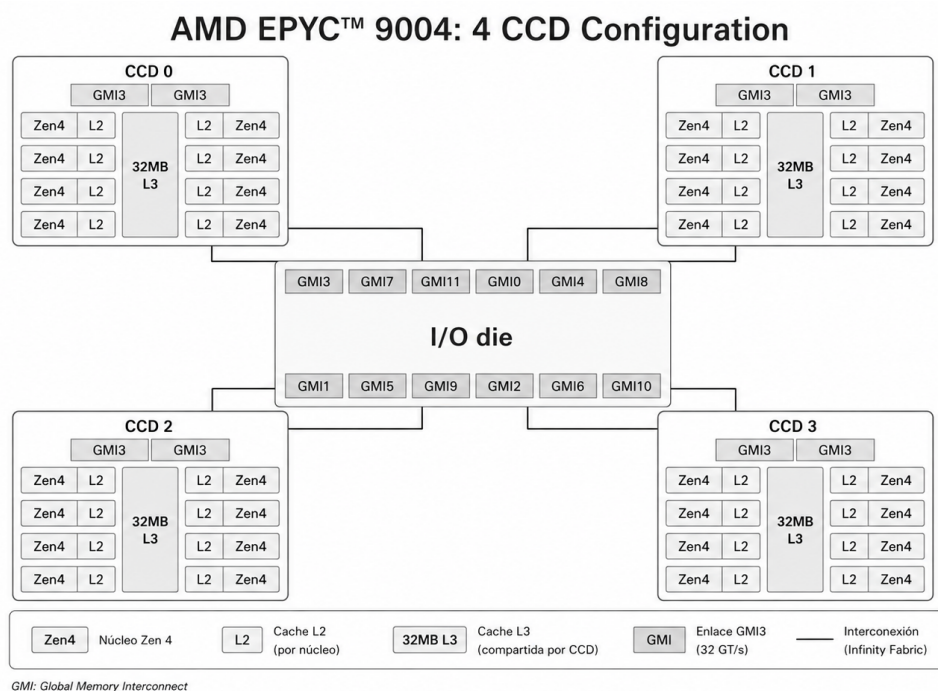


Figura 2.3: Socket de procesador AMD EPYC de la microarquitectura Zen 4. Imagen generada con IA

Comunicación por mecanismos de la caché

La comunicación entre los distintos núcleos se realiza a través de la coherencia de caché, principalmente usando la memoria caché L3 como mecanismo común de memoria. El más popular es el protocolo *Modified Exclusive Shared Invalid (MESI)*, que funciona similar a una máquina de estados que monitoriza el estado de las líneas de caché.

En la Figura 2.4 se muestra un diagrama que indica cómo funciona. En este caso, es un sistema de dos núcleos, que tienen en sus caché el mismo dato de memoria. El núcleo 0 modifica la línea y notifica al otro núcleo sobre el cambio en dicha línea, marcando el dato en el núcleo 1 como

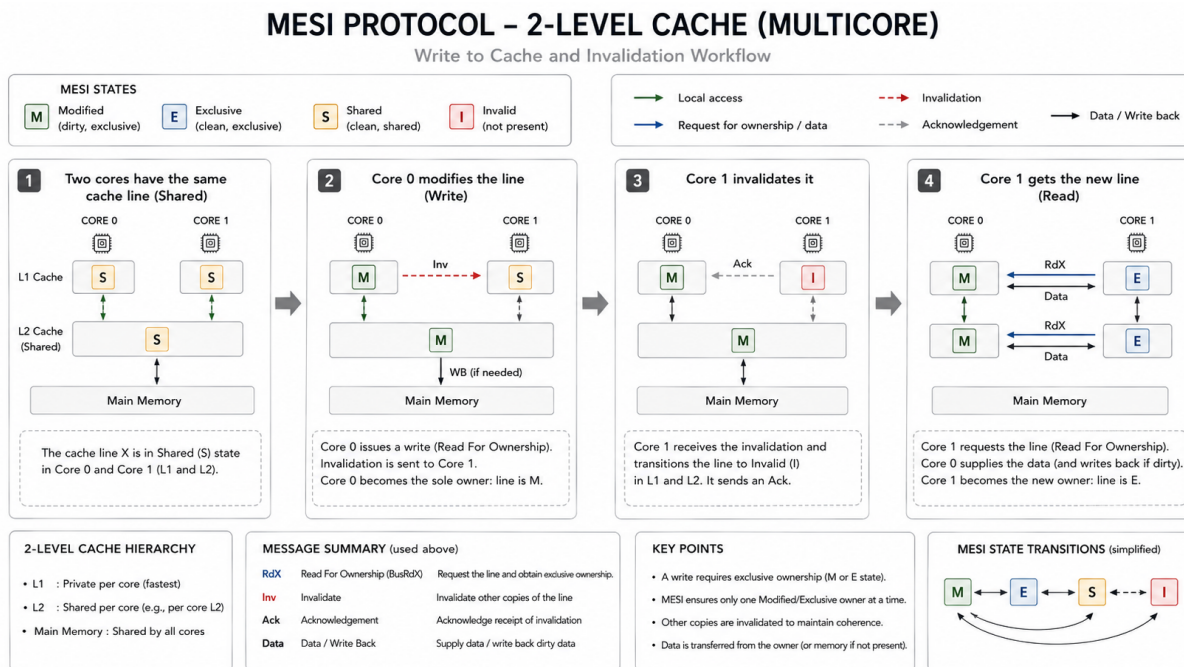


Figura 2.4: Protocolo MESI en sistemas multinúcleo. Imagen generada con IA

inválido. Luego de la invalidación, el núcleo 1 solicita el dato nuevo, dado que el núcleo 0 lo ha modificado, se tiene el dato en modalidad exclusivo, hasta el alguno llegue a leerlo nuevamente, donde se marca como compartido.

El proceso de paso de mensajes en el protocolo MESI implica la utilización de un bus para el envío de notificaciones sobre los estados MESI, tales como invalidaciones y solicitudes exclusivas del dato. Las notificaciones de lectura y escritura son notificadas a todos los núcleos tan pronto ocurren. La técnica que se utiliza se llama *bus snooping*:

1. Cada vez que hay una lectura/escritura, se notifica a todos los núcleos.
2. Otros núcleos escuchan el bus.
3. Si otro núcleo tiene una copia del dato, este puede:
 - Responder con la versión más reciente.
 - Invalidar o actualizar la copia contenida.
 - Forzar un cambio en el registro del MESI.

En el protocolo MESI, es necesario que se escriba el dato en la memoria principal para poder gestionar una actualización. Sin embargo, existen variantes como el *MOESI*, que agregan un estado extra de *owned* sobre el registro, que implica que si un núcleo posee el dato, este puede transferir

el dato sin necesidad de esperar a la escritura. Otra mejora es la basada en directorios, que mapea cuáles núcleos contienen la línea en conflicto, actualizando las líneas de los poseedores y evitando que todos los núcleos sean interrumpidos y actualizados.

Residencia de memoria

En los sistemas de un único *socket*, la memoria principal es compartida y gestionada por el mismo controlador de memoria. Esto implica que todos los núcleos tienen visibilidad de los mismos datos.

Generalmente, cuando un proceso se ejecuta en un núcleo, tiene memorias asociadas, tales como los registros y los contenidos de la memoria caché. Al estado de dichas memorias se le llama *contexto* y define como es el estado de la ejecución de un programa dado.

En la práctica, los sistemas operativos buscan optimizar la calendarización de las tareas para tener el mejor compromiso entre concurrencia y paralelismo. Esto implica que, cuando a una tarea se le agota el tiempo de ejecución, el sistema operativo debe guardar el contexto en memoria y recuperar el contexto de la siguiente tarea, implicando nuevamente cargas en caché. Adicionalmente, sucede un fenómeno que se le denomina *core throttling*, donde una tarea puede brincar de un núcleo a otro, implicando que toda la caché deba ser recargada, implicando múltiples *misses* en el proceso, disminuyendo la eficiencia.

Para evitar esto, usualmente se aplica una técnica que se le denomina *thread affinity*, que fuerza al sistema operativo a utilizar un núcleo en específico para ejecutar un proceso o tarea en específico. Esto reduce los tiempos de carga del contexto.

Dimensionamiento y relevancia del tamaño de las caché

El tamaño de la caché es uno de los factores más importantes en sistemas de computación de gran escala, ya que permite cargar una mayor cantidad de datos de trabajo y disminuir las consultas a la memoria principal.

Han ocurrido casos documentados de procesadores con caché recortada para la disminución de costos que han desembocado en serias degradaciones del rendimiento, aumentando los tiempos de ejecución por encima del doble.

Desde la perspectiva de los tamaños mínimos aceptables, se establece lo siguiente:

- L2: Al menos el tamaño sumado de la L1-I y L1-D.
- L3: Al menos el tamaño de la L2 multiplicado por el número de núcleos.

En el caso del procesador de la arquitectura Zen 3:

- L1-x = 32 KB

- L2 = 512 KB (> 64 KB - 8 veces mayor)
- L3 = 32 MB (> 4 MB - 8 veces mayor)

Sin embargo, dimensionar un procesador con una capacidad alta de memoria caché implica menor densidad dentro del *die*, que significa mayor costo de fabricación.

Sistemas Multisocket

Los sistemas *multisocket* son sistemas que contienen múltiples chips de procesador multinúcleo en el mismo sistema computacional. Es como tomar el complejo computacional descrito en la sección anterior y colocarlo dentro del mismo sistema, comunicándose y colaborando para incrementar aún más el paralelismo.

Este tipo de sistemas se suele encontrar en servidores de centros de datos, servidores de supercomputación y *mainframes*, y están enfocados a servicios de virtualización, manejo de grandes cantidades de datos y a mantener servicios de software de alta demanda, como sitios web de alto tráfico.

A diferencia de los sistemas multinúcleo, los sistemas de procesamiento multisocket tienen la caché L3 aislada (por *socket*) y tienen un manejador de memoria independiente (también por *socket*), lo que permite que cada complejo de núcleos tenga independencia e inclusive su propia memoria principal.

Sin embargo, esta independencia involucra problemas serios, como fragmentación del espacio de direcciones y de datos, que involucran penalizaciones si los datos no son reservados en el banco donde van a ser procesados. Asimismo, mover datos dentro de la memoria principal puede implicar la transferencia de los datos de un *socket* a otro, mediante sistemas de mensajería.

La Figura 2.5 ilustra un sistema de procesamiento *multisocket*. En este se destaca la presencia de dos procesadores y un mecanismo de interconexión que puede ser mediante un bus o una comunicación en malla entre los mismos procesadores. Asimismo, se ilustra que cada procesador tiene su banco de memoria principal, gestionado de forma independiente por cada procesador.

En cuanto al manejo de los datos, este se realiza de la misma manera a nivel de caché, donde el sistema de comunicación interprocesador permite el reenvío de mensajes entre cachés, extendiendo el mecanismo que puede estar basado en MESI o uno más avanzado.

Un aspecto importante es el medio de comunicación entre procesadores. Este funciona como un mecanismo de mensajería sobre el estado de cada procesador, coordinación y, además, intercambio de datos. En el caso de los procesadores Intel, el sistema de comunicación se llama *Ultra Path Interconnect* (UPI), y en AMD es el *Infinity Fabric*, que sirve igualmente para comunicación intrasocket. Estos dos mecanismos de comunicación permiten transferir datos a tasas mayores a 150 GB/s (1.2 Tb/s).

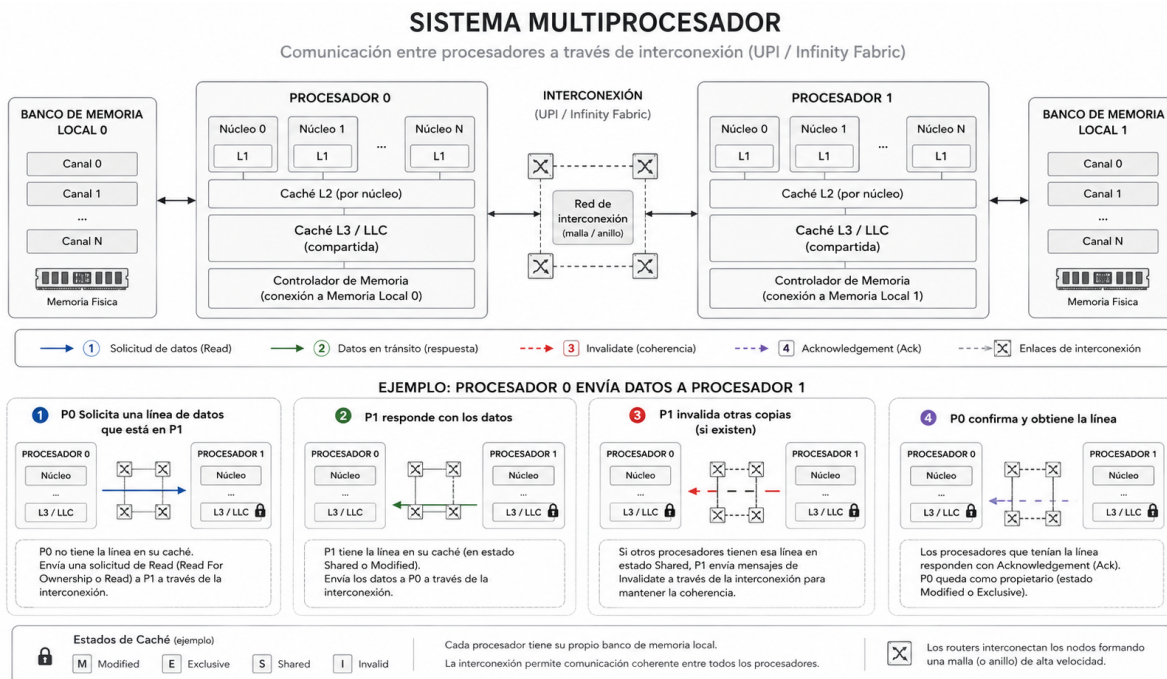


Figura 2.5: Sistema de procesamiento *multisocket*. Este sistema debe extender el mecanismo de coherencia de memoria y debe habilitar un canal de intercambio de datos. Imagen generada con IA

Con respecto a AMD, en la Figura 2.6, en organizaciones recientes, el socket está compuesto por múltiples *chiplets*, que contienen un CCD. Estos *chiplets* se comunican a través de *Infinity Fabric*, de la misma manera en la que se comunican dos procesadores separados.

NUMA: Non-Uniform Memory Access

En sistemas multiprocesador, un escenario posible es que un núcleo de procesador intente acceder a un dato que se encuentre en otro *socket*. A nivel de programación, el sistema operativo se encarga de hacer transparente y abstracto el proceso, donde el programador no se da cuenta de esta transferencia. Sin embargo, a nivel de hardware, acceder a dicho dato implica una serie de transacciones, desde buscar dentro de la caché, enviar consultas a través del bus de interconexión inter-socket hasta que el dato se transfiere de la memoria del *socket* fuente al *socket* destino.

El manejo de estos casos se hace a través de NUMA, que es un mecanismo junto con un controlador de hardware que permite discernir entre la localización de datos dentro del mismo *socket* y en distintos *sockets*. En el caso de los procesadores AMD EPYC, esto puede llegar a suceder inclusive dentro del mismo *socket*, ya que los núcleos de procesamiento se agrupan en *clusters* que se comunican a través de *Infinity Fabric*.

Como se ilustra en la Figura 2.7, en los casos donde los datos se encuentren de forma local, la

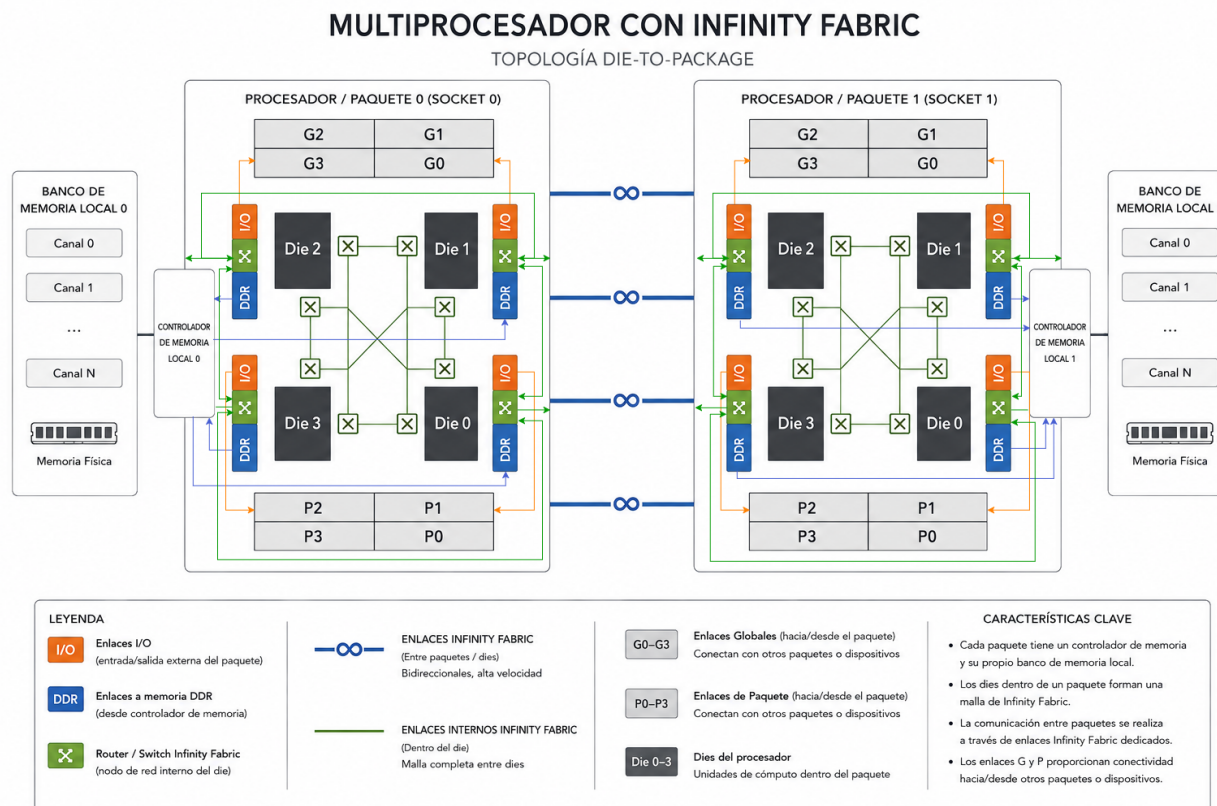


Figura 2.6: AMD Zen. Tiene *chiplets* que se interconectan como si fueran procesadores *multisocket*. Imagen generada con IA y editada por el autor.

latencia es mucho menor que en aquellos casos donde se encuentren en otro *cluster* o *socket*, donde la latencia suele ser proporcional a la distancia a la que se encuentra el dato almacenado del núcleo de procesamiento.

Optimización de transferencia de datos

Los sistemas con procesamiento multisocket suelen ejecutar cargas de trabajo que seccionan y reparte el trabajo entre todos los núcleos, donde cada uno ejecuta tareas equivalentes pero sobre conjuntos de datos distintos.

Cuando se realiza una reserva de memoria con `malloc`, la memoria se reserva en el banco de memoria asociado al *socket* en el que se está ejecutando la reserva. Por ello, surge la primera consideración:

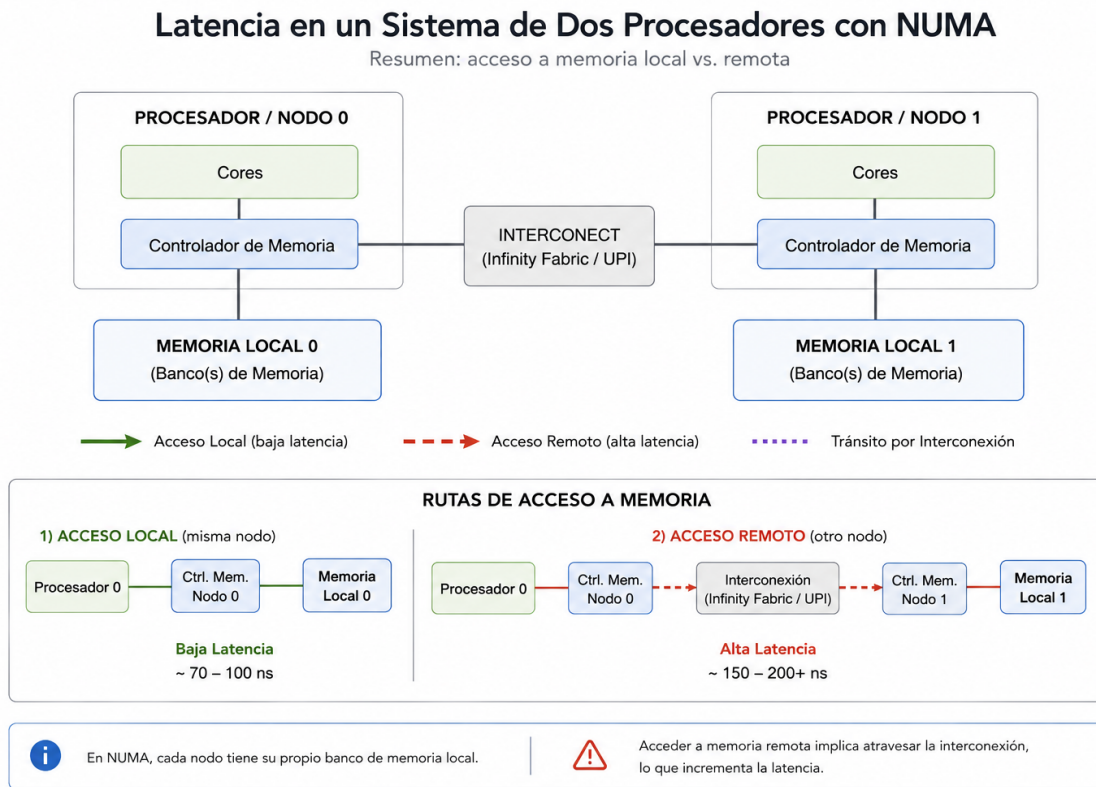


Figura 2.7: Ilustración de NUMA. La ruta más corta tiene menor latencia. Imagen generada con IA

Consideración

La memoria se debe reservar en el mismo núcleo/socket donde se va a ejecutar el trabajo.

Posteriormente, es necesario considerar el fenómeno del *thread throttling*, donde la tarea brinca de núcleo a núcleo durante la ejecución por decisión arbitraria del sistema operativo. Para ello, se debe restringir la ejecución de la tarea en un núcleo en específico. Esto permite:

- Evitar que la tarea brinque de un núcleo a otro, ocasionando problemas de migración de contexto y de enfriamiento de la caché (que pierda sus datos).
- Evitar que la tarea brinque a otro *socket* o CCD (en el caso de la microarquitectura Zen), añadiendo problemas de latencia por NUMA.

Por ello, se sigue la segunda consideración:

Consideración

La tarea o hilo debe vincularse a un único núcleo para evitar el *throttling*.

Para ejemplificar estos casos, podemos considerar primero un caso donde se deja al OS la libertad de decidir los núcleos:

Listing 2.1: Código Ingenuo

```
// Hilo trabajador
static void *worker(void *arg) {
    thread_arg_t *thread_arg = (thread_arg_t *)arg;

    size_t elements = thread_arg->elements;
    double *memory = thread_arg->memory;

    // Carga del 100%.
    cpu_burn(memory, elements);
    return NULL;
}

int main() {
    // ...
    // Reserva de memoria en el hilo main
    for (int i = 0; i < NUM_THREADS; i++) {
        double *memory = NULL;

        int ret = posix_memalign((void **)&memory, 64, MEMORY_SIZE);
        size_t elements = MEMORY_SIZE / sizeof(double);

        args[i].thread_id = i;
        args[i].memory_size = MEMORY_SIZE;
        args[i].elements = elements;
        args[i].memory = memory;
    }

    // Invocación
    //...
    for (int i = 0; i < NUM_THREADS; i++) {
        int ret = pthread_create(&threads[i], NULL, worker, &args[i]);

        if (ret != 0) {
            fprintf(stderr, "Error creando thread %d\n", i);
            return EXIT_FAILURE;
        }
    }
    //...
    return 0;
}
```

Ahora, podemos mejorar la reserva de memoria para que sea asignada al núcleo y fijar el núcleo lógico (que abstrae el computador para que sea transparente al programador) donde se va a ejecutar. Para fijar el hilo a un núcleo en específico `pthread_setaffinity_np`.

Listing 2.2: Código Mejorado

```
// Metodo para asignar la afinidad
static int pin_thread_to_cpu(int cpu_id) {
    cpu_set_t cpuset;
    CPU_ZERO(&cpuset);
    CPU_SET(cpu_id, &cpuset);
    return pthread_setaffinity_np(pthread_self(), sizeof(cpuset), &cpuset);
}

// Hilo trabajador
static void *worker(void *arg) {
    thread_arg_t *thread_arg = (thread_arg_t *)arg;
    int id = thread_arg->thread_id;
    int cpu = thread_arg->cpu_id;
    size_t memory_size = thread_arg->memory_size;

    pin_thread_to_cpu(cpu);
    double *memory = NULL;

    // Reserva dentro del trabajador
    posix_memalign((void **)&memory, 64, memory_size);
    size_t elements = memory_size / sizeof(double);

    // Trabajo
    cpu_burn(memory, elements);

    // Liberar memoria
    free(memory);
    return NULL;
}

int main() {
    // ...
    // Invocaci\on
    for (int i = 0; i < NUM_THREADS; i++) {
        int ret = pthread_create(&threads[i], NULL, worker, &args[i]);

        if (ret != 0) {
            fprintf(stderr, "Error creando thread %d\n", i);
            return EXIT_FAILURE;
        }
    }
    //...
    return 0;
}
```

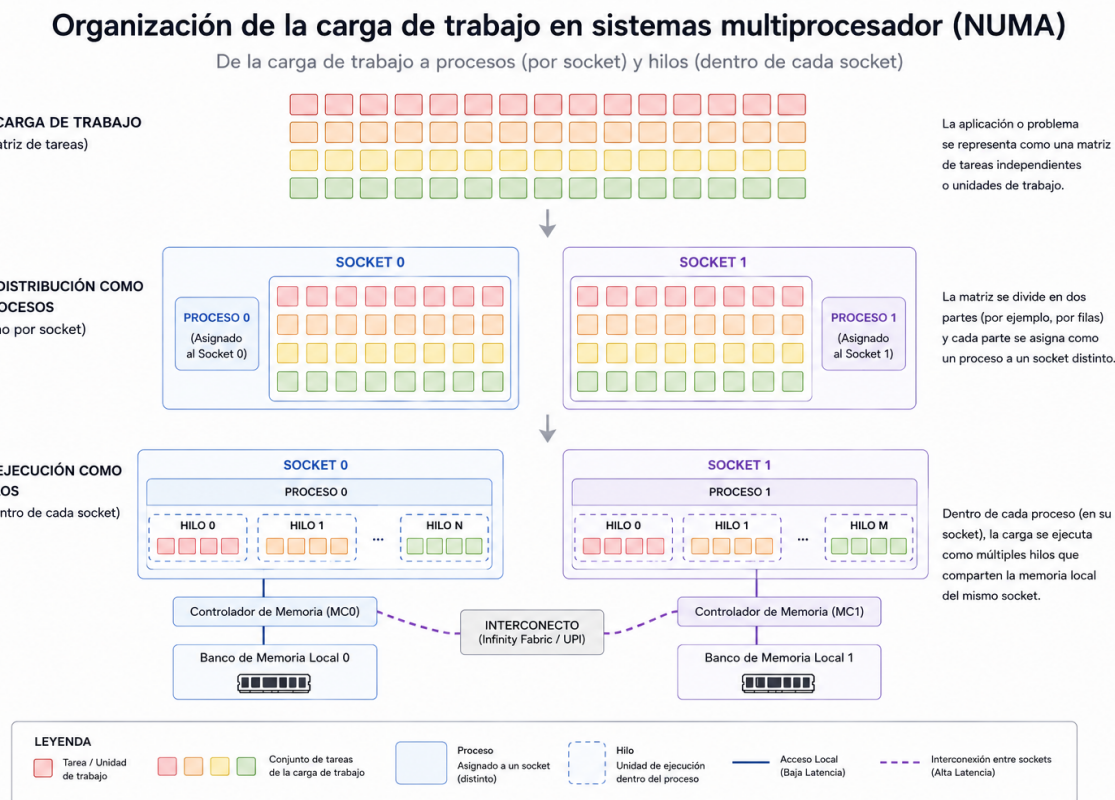


Figura 2.8: Repartición de la carga de trabajo en la abstracción de proceso/hilo y el hardware respectivo. Imagen generada con IA

Otra consideración importante es la organización del sistema en cuanto a su distribución de núcleos. Esta recomendación es particularmente importante para determinar si la carga de trabajo debe repartirse como proceso o como hilo, tal como se ilustra en la Figura 2.8. Generalmente, hay una regla empírica:

- Ejecutar procesos en *sockets* distintos.
- Ejecutar hilos dentro del mismo *socket*.

Esto ya que los procesos tienen su propia abstracción o espacio de direcciones virtual, que es conveniente que quede dentro de un único banco de memoria por cuestiones de latencia. Por otra parte, los hilos de un proceso usan el mismo espacio de direcciones, por lo que las transferencias intrasocket suelen tener *menos overhead*. Por ello:

Consideración

Repartir la carga de trabajo tal que un proceso quede en un socket, y que los hilos queden dentro de un mismo socket.

Obviar esta consideración incurre en copias de memoria innecesarias, empeorando la latencia del programa.

2.2. Escalabilidad y Retos Actuales

Como se vio en el capítulo anterior, la escalabilidad de un sistema está acotada asintóticamente por la Ley de Amdahl, según la cual la aceleración de un sistema disminuye a medida que aumenta el número de elementos de procesamiento. En la *Historia de los 1000 núcleos*, publicado en el 2022, es posible apreciar el fenómeno descrito [5].

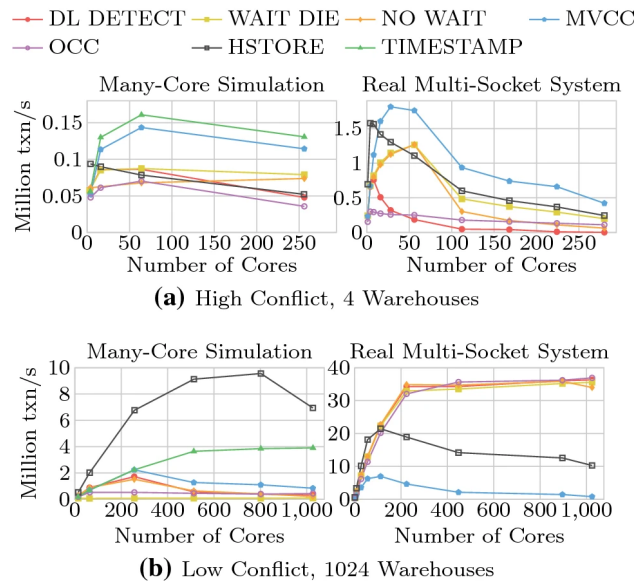


Figura 2.9: La historia de los 1000 núcleos. Ilustración del escalamiento. Obtenido de [5]

Más concretamente, el tiempo de un proceso puede determinarse principalmente por dos componentes: 1) el tiempo de cómputo y 2) el tiempo de comunicación. Una de las técnicas más comunes en la computación heterogénea es la ocultación de la latencia de comunicación, transmitiendo datos mientras se ejecuta carga de trabajo en todas las unidades de procesamiento. Para ello, se recurren a técnicas como (ilustradas en la Figura 2.10):

- Ping-pong buffers
- Ring-buffers

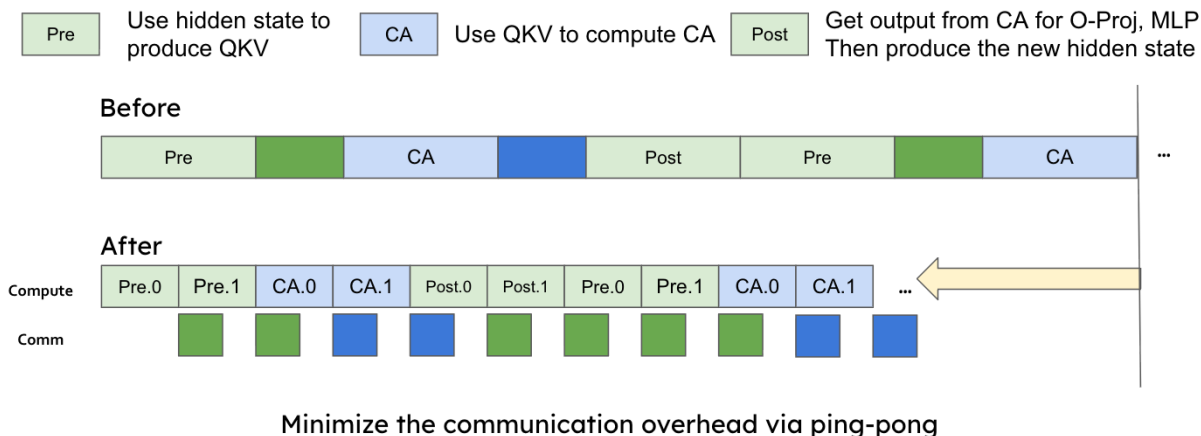
Ping-Pong Schedule: hide communication cost

Figura 2.10: Uso de ping-pong buffers para el ocultamiento de la latencia. Un buffer se transmite mientras que se procesa otro bloque de memoria. Obtenido de [6]

Sin embargo, el problema recrudece cuando la comunicación sale de un socket, aumentando en órdenes de magnitud. La Tabla 2.1 ilustra esto en particular. Se puede notar que mientras los datos se encuentren dentro del socket, el tiempo de latencia ronda a lo sumo decenas de nanosegundos. En el momento en el que sale, aumenta a las centenas. Si sale del sistema (nodo), aumenta a los miles de nanosegundos [7].

Tabla 2.1: Latency Scale from CPU Cache to External Networks. Basado en [7]

Boundary Level	Typical Latency	Primary Hardware/Protocol
On-Core (L1/L2 Cache)	1–4 ns (10^{-9} s)	SRAM, Internal Interconnect
On-Socket (L3 Cache)	10–20 ns (10^{-9} s)	Shared SRAM Ring/Mesh
Outside CPU Socket	50–100+ ns (10^{-9} s)	Intel UPI, AMD Infinity Fabric
Main Memory (Local RAM)	60–80 ns (10^{-9} s)	DDR4/DDR5 Memory Controller
Inside Data Center	1–10 μ s (10^{-6} s)	100GbE Switches, NVMe-oF, RDMA
Outside Network (WAN)	50–150+ ms (10^{-3} s)	TCP/IP, Fiber Optic, Edge Routers

Por ello, para optimizar una carga de trabajo, debe considerarse la porción de código paralelizable, la susceptibilidad de comunicación y la organización del sistema. Sin embargo, es importante tomar en cuenta que el límite es asintótico y que cada carga de trabajo tiene sus propias proporciones paralelizables. Ante esta situación, una regla empírica que se utiliza es que no se debe paralelizar más allá del codo de la gráfica de escalamiento.

Consideración

El escalamiento de una carga de trabajo depende de la organización de un sistema, su programación en cuanto a accesos a datos y porciones de código paralelizables. Se considera que una carga de trabajo no debe escalarse más allá del codo de la gráfica de escalamiento o menor a 50 % de eficiencia.

Dentro de los retos actuales, la inteligencia artificial (IA) ha traído altibajos con respecto al uso de la CPU, favoreciendo el mercado de unidades de cómputo para gráficas (GPU). Sin embargo, el advenimiento de los agentes de IA ha ubicado nuevamente a los CPUs como unidades de procesamiento que permiten organizar a los agentes. La infografía de la Figura 2.11 ilustra una serie de desafíos nuevos entorno a los agentes de IA.

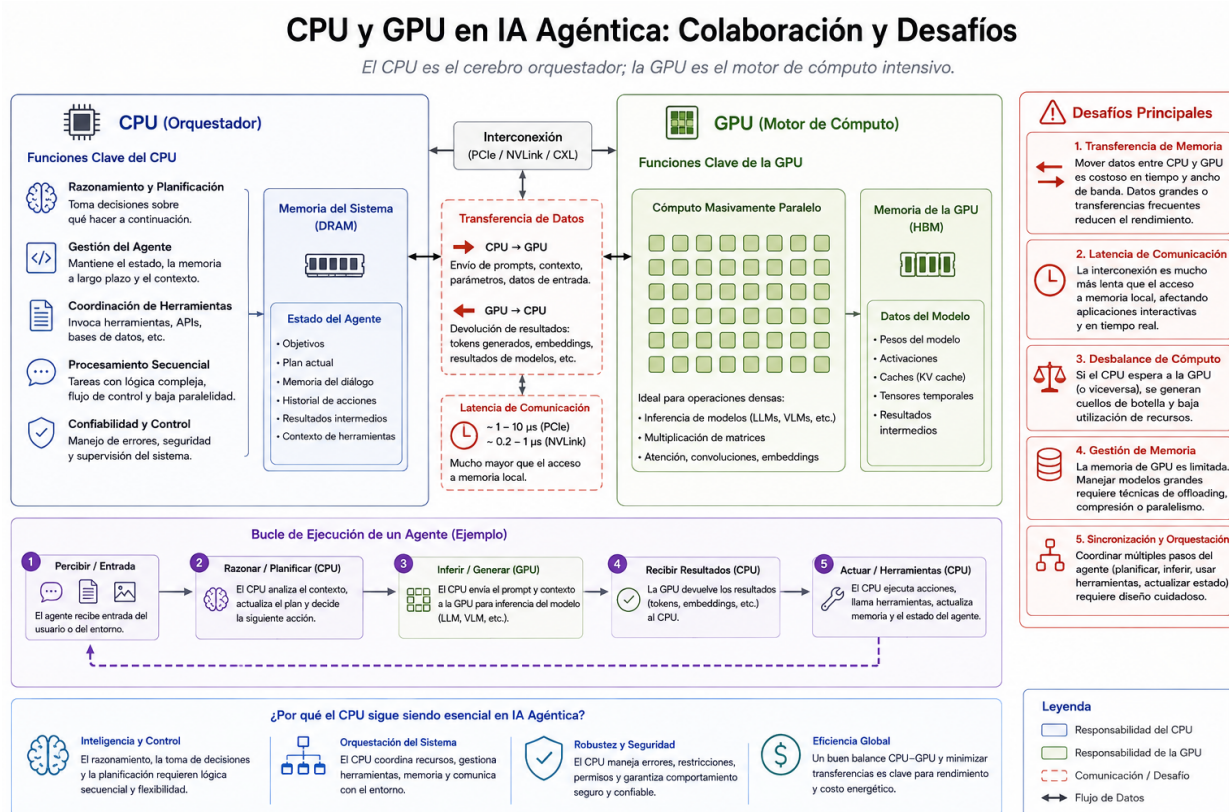


Figura 2.11: Retos de la IA agéntica. Imagen generada con IA

Dentro de los principales desafíos se pueden enumerar los enlaces de intercomunicación entre CPU-GPU. Tradicionalmente, se usaba PCIe para dicha comunicación. Empresas como NVIDIA se encuentran trabajando en la integración de chips híbridos comunicados a través de NVLink, que es un protocolo de comunicación de alta velocidad que permite tener coherencia de memoria. Otro aspecto importante es la cantidad de núcleos, ya que permite formular estrategias en paralelo y

ejecutar instrucciones, mientras que el GPU ejecuta el modelo de razonamiento más concretas. Por ello, se puede definir que los CPUs están modelándose en las aristas de:

- Más núcleos de procesamiento.
- Mejor comunicación CPU-GPU.
- Mejor acceso a la memoria.

Esto corresponde a los nuevos retos, donde el CPU comienza a tomar relevancia nuevamente en el mundo de la IA.

2.3. Programación de Sistemas Multiprocesador

La programación de sistemas multiprocesador puede llegar a tener altos niveles de abstracción, como el uso de lenguajes interpretados como Python, que son capaces de ejecutar tareas en varios núcleos y varios *sockets*. Sin embargo, la abstracción lleva el costo asociado a decisiones poco optimizadas o que tratan de generalizarse en todas las plataformas, favoreciendo la portabilidad.

Pregunta

¿Qué es y qué implica la portabilidad de un código?

Esto implica que las plataformas no son utilizadas al 100 %. Inclusive, subir a C desde ensamblador también tiene un costo asociado. La idea de esta sección es guiar a través de los distintos conceptos de un código, entendiendo su trasfondo y obteniendo ideas de cómo abordarlo desde la perspectiva de optimización, aplicando conceptos de este y el capítulo anterior.

Composición de un programa

El proceso de arranque de un proceso implica distintos procesos, particularmente en aplicaciones grandes, que impliquen el uso de bibliotecas. Generalmente, un ejecutable (programa ejecutable) se compila a partir de distintas bibliotecas que proporcionan funcionalidad, siendo `libc` una de ellas.

De manera simplificada, este proceso puede representarse como:

Código fuente → Compilación → Código objeto → Enlazado → Ejecutable → Carga → Ejecución

En sistemas modernos, este proceso involucra no solamente al compilador, sino también al enlazador (*linker*), las bibliotecas del sistema, el sistema operativo y el cargador (*loader*).

Considérese el siguiente programa escrito en C:

```
#include <stdio.h>

int main(void) {
```

```
printf("Hola mundo\n");  
return 0;  
}
```

El compilador transforma el código fuente en instrucciones correspondientes a la arquitectura destino, por ejemplo, x86-64, ARM o RISC-V. Sin embargo, el resultado de la compilación no necesariamente constituye todavía un programa ejecutable. Normalmente, cada archivo fuente se transforma primero en un **archivo objeto** (*object file*). Por ejemplo:

```
gcc -c main.c -o main.o
```

Esto genera el fragmento del código traducido en ensamblador directamente, pero no tiene el enlace con la biblioteca de C ni su punto de arranque definido. En una síntesis, contiene:

- Código máquina generado para las funciones compiladas.
- Datos y constantes utilizados por el programa.
- Una tabla de símbolos.
- Información necesaria para la reubicación del código.
- Referencias a símbolos externos que todavía no han sido resueltos.

Por ejemplo, la función `printf()` no está implementada dentro de `main.c`. El archivo objeto contiene una referencia a esta función que deberá ser resuelta posteriormente.

El **enlazador** (*linker*) combina uno o varios archivos objeto y resuelve las referencias existentes entre ellos. Además, incorpora las bibliotecas necesarias para construir el programa final. De manera conceptual:

$$\text{main.o} + \text{bibliotecas} \xrightarrow{\text{linker}} \text{ejecutable}$$

Una de las responsabilidades principales del enlazador es la **resolución de símbolos**. Si un archivo objeto utiliza una función definida en otro archivo o biblioteca, el enlazador debe determinar dónde se encuentra su implementación. Por ejemplo:

```
main.o: main() referencia a printf()  
libc: printf()
```

El enlazador debe relacionar la referencia a `printf()` con su implementación correspondiente.

Bibliotecas

Una biblioteca es una colección de funciones y recursos reutilizables que pueden ser utilizados por diferentes programas. Las bibliotecas permiten evitar la duplicación de código y facilitan la modularización del software.

Existen principalmente dos formas de incorporar una biblioteca a un programa:

- Enlazado estático (*static linking*).
- Enlazado dinámico (*dynamic linking*).

Bibliotecas estáticas

Una biblioteca estática contiene código objeto que puede incorporarse directamente dentro del ejecutable durante el proceso de enlazado.

En sistemas GNU/Linux, estas bibliotecas normalmente utilizan la extensión `.a`. Por ejemplo: `libexample.a`. Durante el enlazado estático, el código requerido de la biblioteca es incorporado al ejecutable:

$$\text{main.o} + \text{libexample.a} \rightarrow \text{programa}$$

Como consecuencia, el ejecutable contiene el código necesario para utilizar las funciones provenientes de la biblioteca, por lo que reduce las dependencias externas con el sistema y futuros problemas por dependencias no instaladas. Sin embargo, esto implica que el código tenga mayor tamaño.

Bibliotecas dinámicas

En el enlazado dinámico, el código de la biblioteca no se incorpora completamente al ejecutable. En su lugar, el ejecutable mantiene información sobre las bibliotecas que deben estar disponibles durante la ejecución. En sistemas GNU/Linux, estas bibliotecas normalmente poseen nombres como: `libexample.so`. Conceptualmente:

$$\text{programa} + \text{libexample.so} \xrightarrow{\text{tiempo de ejecución}} \text{Programa en ejecución}$$

Se puede notar que una de las principales diferencias es que la biblioteca estática se incrusta dentro del programa, mientras que en la dinámica, el programa declara que lo necesita y el sistema operativo busca el `.so` en los directorios de bibliotecas instaladas en el sistema. Esto implica, además, que la biblioteca pueda ser compartida, disminuyendo el tamaño de los archivos en el sistema, actualizaciones independientes y menor tamaño en los binarios, a costa de la posibilidad de errores de dependencias.

El archivo ejecutable

Después del proceso de enlazado se obtiene un **archivo ejecutable**. Este archivo contiene el código y la información necesaria para que el sistema operativo pueda crear un nuevo proceso. El formato del ejecutable depende del sistema operativo. Algunos formatos comunes son:

- **ELF (*Executable and Linkable Format*)**: ampliamente utilizado en GNU/Linux y otros sistemas tipo Unix.
- **PE (*Portable Executable*)**: utilizado principalmente por Windows.

Un ejecutable contiene diferentes regiones con propósitos específicos. De forma simplificada, pueden encontrarse:

- **.text**: código ejecutable del programa.
- **.rodata**: constantes y datos de solo lectura.
- **.data**: variables globales y estáticas inicializadas.
- **.bss**: variables globales y estáticas no inicializadas.
- **Tabla de símbolos**: información sobre funciones y variables.
- **Información de reubicación**: utilizada para determinar las direcciones finales de diferentes elementos.
- **Información de enlazado dinámico**: referencias a bibliotecas compartidas cuando corresponda.

El ejecutable también especifica un **punto de entrada** (*entry point*), que indica la dirección donde debe comenzar la ejecución del programa. Es importante destacar que el punto de entrada de un ejecutable C normalmente **no corresponde directamente a la función `main()`**. Antes de invocar `main()`, debe ejecutarse código de inicialización proporcionado por el entorno de ejecución dentro del sistema operativo, que permite inicializar, arrancar y buscar los enlaces [8], tal como se ilustra en la Figura 2.12.

Compilación de un programa

El compilador no solamente traduce un programa desde un lenguaje de alto nivel hacia instrucciones de máquina. Durante este proceso también puede transformar el código con el objetivo de mejorar características como el tiempo de ejecución, el tamaño del ejecutable o el aprovechamiento de los recursos de la microarquitectura.

Considérese, por ejemplo, la siguiente operación:

```
int x = 5 * 4;
```

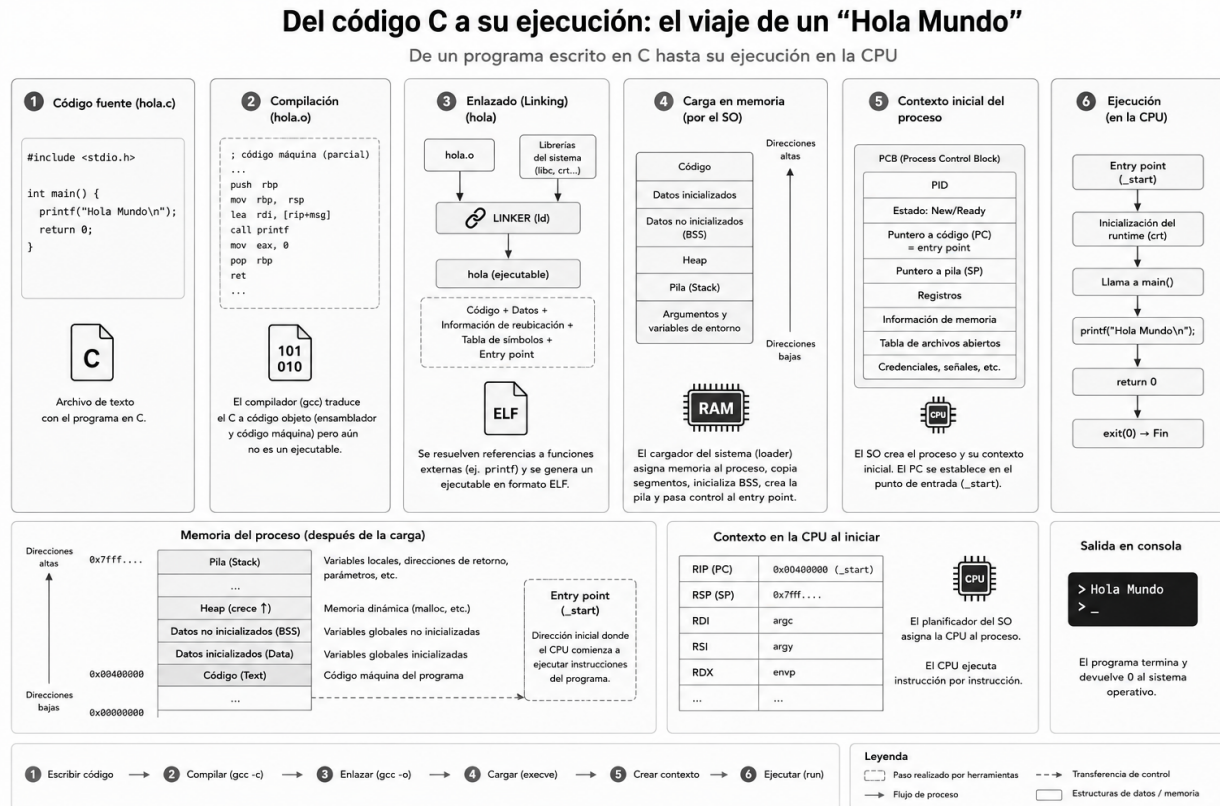


Figura 2.12: Proceso de arranque de un programa. Imagen asistida con IA

Aunque el programador expresa una multiplicación, el compilador puede determinar durante la compilación que el resultado es constante y sustituir la operación por:

```
int x = 20;
```

Esta transformación evita realizar la multiplicación durante la ejecución. A este tipo de optimización se le conoce como *constant folding*.

Las optimizaciones realizadas por un compilador pueden actuar sobre diferentes aspectos del programa, entre ellos:

- Eliminación de operaciones innecesarias.
- Reducción del número de accesos a memoria.
- Mejor utilización de los registros del procesador.
- Transformación y reorganización de ciclos.

- Vectorización de operaciones.
- Reorganización de instrucciones.
- Eliminación de funciones o datos que nunca son utilizados.
- Sustitución de operaciones por otras de menor costo.

En GCC, el usuario puede controlar muchas de estas transformaciones mediante las opciones de optimización `-O`. Los niveles más utilizados están descritos en la Tabla 2.2.

Por ejemplo, un programa puede compilarse sin optimizaciones mediante:

```
gcc -O0 program.c -o program
```

y utilizando optimizaciones agresivas mediante:

```
gcc -O3 program.c -o program
```

Es importante destacar que un nivel de optimización mayor **no garantiza automáticamente un programa más rápido**. El resultado depende del algoritmo, la arquitectura, el patrón de acceso a memoria y las características de la carga de trabajo. Por esta razón, el rendimiento debe medirse experimentalmente.

Optimizaciones de bajo nivel

En este apartado estaremos abordando algunas técnicas de optimización de código mononúcleo, que solo utiliza un núcleo de procesamiento.

Optimizaciones comunes

Consideración

Para esta sección, es necesario conocer C y su funcionamiento a nivel de mapeo C-Ensamblador.

Los niveles `-O1`, `-O2` y `-O3` habilitan diferentes transformaciones internas. Algunas de las más importantes para comprender la relación entre compiladores y arquitectura de computadores se describen a continuación.

Propagación y plegado de constantes

Cuando el valor de una expresión puede determinarse durante la compilación, el compilador puede calcularlo anticipadamente. Por ejemplo:

```
int x = 10;
int y = x * 4;
```

Tabla 2.2: Principales niveles de optimización disponibles en GCC.

Bandera	Objetivo	Descripción
-O0	Sin optimización	Reduce las transformaciones realizadas sobre el código. Facilita la depuración y normalmente produce una correspondencia más directa entre el código fuente y las instrucciones generadas.
-O1	Optimización básica	Habilita optimizaciones relativamente sencillas buscando mejorar el rendimiento y el tamaño del código sin aumentar excesivamente el tiempo de compilación.
-O2	Optimización general	Habilita un conjunto amplio de optimizaciones que generalmente mejoran el rendimiento sin realizar transformaciones particularmente agresivas que incrementen considerablemente el tamaño del ejecutable. Es una opción común para software de producción.
-O3	Máximo rendimiento	Añade optimizaciones más agresivas, particularmente sobre ciclos, vectorización y funciones. Puede incrementar el tamaño del código y no garantiza que el programa sea siempre más rápido que con -O2.
-Os	Tamaño	Optimiza buscando reducir el tamaño del ejecutable, evitando ciertas transformaciones que podrían aumentarlo. Es particularmente útil en sistemas embebidos con restricciones de memoria.
-Og	Depuración	Habilita optimizaciones compatibles con una buena experiencia de depuración. Representa un compromiso entre -O0 y los niveles de optimización superiores.
-Ofast	Rendimiento agresivo	Habilita optimizaciones agresivas que pueden relajar ciertos requisitos semánticos del lenguaje. Debe utilizarse cuidadosamente, particularmente en cálculos de punto flotante.

puede transformarse conceptualmente en:

```
int y = 40;
```

De esta manera, operaciones que originalmente estaban presentes en el programa pueden desaparecer completamente del código máquina.

Common Subexpression Elimination

Si una expresión equivalente se calcula repetidamente y sus operandos no han cambiado, el compilador puede reutilizar un resultado previamente calculado. Por ejemplo:

```
a = (x + y) * 2;  
b = (x + y) * 4;
```

puede aprovechar un único cálculo de:

$x + y$

reduciendo la cantidad total de operaciones.

Inlining de funciones

Una llamada a una función introduce ciertas operaciones adicionales asociadas con la transferencia de control y el manejo de sus argumentos. Para funciones pequeñas, el compilador puede sustituir la llamada por el cuerpo de la función. Por ejemplo:

```
int square(int x) {  
    return x * x;  
}  
  
y = square(a);
```

puede convertirse conceptualmente en:

```
y = a * a;
```

Esta técnica se conoce como *function inlining*.

El *inlining* puede reducir el costo de llamadas a funciones, ya que no hay saltos en la memoria de instrucciones, permitiendo una estructura más amigable con la caché L1-I. También permitir optimizaciones adicionales sobre el código resultante. Sin embargo, el tamaño del ejecutable si se utiliza excesivamente. GCC puede controlar esta transformación mediante opciones como `-finline-functions`.

Desenrollado de ciclos

El **desenrollado de ciclos** (*loop unrolling*) reduce la cantidad de operaciones utilizadas para controlar un ciclo. Considere:

```
for (int i = 0; i < 4; ++i)
    a[i] = b[i] + c[i];
```

Una transformación conceptual podría producir:

```
a[0] = b[0] + c[0];
a[1] = b[1] + c[1];
a[2] = b[2] + c[2];
a[3] = b[3] + c[3];
```

Con ello se reduce la cantidad de instrucciones relacionadas con incrementar el contador, evaluar la condición y realizar la bifurcación. Además, el desenrollado puede exponer más operaciones independientes al procesador y facilitar otras optimizaciones. Sin embargo, también puede incrementar considerablemente el tamaño del código. GCC dispone, entre otras, de la opción:

```
-funroll-loops
```

Vectorización

Los procesadores modernos poseen extensiones SIMD (*Single Instruction, Multiple Data*) capaces de aplicar una misma operación sobre varios elementos simultáneamente:

```
for (int i = 0; i < N; ++i)
    c[i] = a[i] + b[i];
```

Una implementación escalar puede procesar un elemento por instrucción:

$$a_0 + b_0, a_1 + b_1, a_2 + b_2, a_3 + b_3, \dots$$

Una implementación vectorizada puede procesar múltiples elementos mediante una sola instrucción SIMD:

$$(a_0, a_1, a_2, a_3) + (b_0, b_1, b_2, b_3)$$

GCC puede realizar automáticamente muchas de estas transformaciones mediante sus mecanismos de **autovectorización**. Opciones relacionadas incluyen:

```
-ftree-vectorize
-fopt-info-vec
-fopt-info-vec-missed
```

Las últimas opciones son particularmente útiles para estudiar el comportamiento del compilador, ya que permiten obtener información sobre los ciclos que fueron vectorizados y aquellos que no pudieron serlo.

Optimización para la microarquitectura

Además de optimizar el código de forma general, GCC puede generar instrucciones específicas para el procesador donde se ejecutará el programa. Una opción importante es:

```
-march=native
```

Esta opción permite utilizar las características disponibles en la máquina donde se realiza la compilación, incluyendo extensiones particulares del conjunto de instrucciones. Por ejemplo:

```
gcc -O3 -march=native program.c -o program
```

puede permitir que GCC utilice instrucciones SIMD u otras extensiones disponibles en el procesador. Otra opción relacionada es:

```
-mtune=native
```

Conceptualmente, existe una diferencia importante:

- **-march** determina qué **instrucciones** puede utilizar el compilador.
- **-mtune** modifica cómo se organiza el código para favorecer las características de una determinada **microarquitectura**, sin necesariamente cambiar el conjunto base de instrucciones permitido.

Por esta razón, un ejecutable compilado con **-march=native** puede no ser compatible con procesadores que no soporten las mismas extensiones.

Consideración

Este tipo de optimizaciones no son portables.

Optimización entre unidades de compilación

Normalmente GCC compila cada archivo fuente de manera relativamente independiente:

```
gcc -c a.c  
gcc -c b.c  
gcc a.o b.o -o program
```

Esto limita ciertas optimizaciones, pues durante la compilación de **a.c** el compilador puede no disponer de suficiente información sobre las funciones implementadas en **b.c**.

La optimización durante el enlazado, denominada *Link-Time Optimization* (LTO), permite realizar optimizaciones considerando varias unidades de compilación simultáneamente.

Puede habilitarse mediante:

```
gcc -O3 -flto a.c b.c -o program
```

LTO puede facilitar optimizaciones como el *inlining* entre diferentes archivos, eliminación de código no utilizado y propagación de información entre funciones.

Optimización guiada por perfil

No todas las partes de un programa se ejecutan con la misma frecuencia. Algunas funciones y ciclos constituyen las regiones críticas del programa, mientras que otras se ejecutan ocasionalmente.

La **optimización guiada por perfil** (*Profile-Guided Optimization*, PGO) utiliza información obtenida durante ejecuciones reales del programa para orientar las decisiones del compilador.

El proceso general consiste en:

1. Compilar el programa incorporando instrumentación.
2. Ejecutarlo utilizando una carga de trabajo representativa.
3. Recopilar información sobre su comportamiento.
4. Volver a compilar utilizando dicha información.

Con GCC puede realizarse, de forma simplificada, mediante:

```
gcc -O3 -fprofile-generate program.c -o program
./program
```

```
gcc -O3 -fprofile-use program.c -o program
```

El compilador puede utilizar el perfil para tomar mejores decisiones sobre *inlining*, organización del código y bifurcaciones, entre otras transformaciones.

Punto flotante y `-ffast-math`

Las operaciones de punto flotante poseen propiedades particulares que pueden limitar ciertas transformaciones matemáticas. Por ejemplo, debido al redondeo, la suma de números de punto flotante no es estrictamente asociativa:

$$(a + b) + c \neq a + (b + c) \quad (2.1)$$

para algunos valores representables.

La opción:

```
-ffast-math
```

permite que GCC realice transformaciones más agresivas sobre operaciones de punto flotante asumiendo condiciones menos estrictas que las requeridas normalmente.

Esto puede mejorar el rendimiento y facilitar la vectorización, pero puede modificar los resultados numéricos o el comportamiento frente a casos especiales. Por esta razón, debe utilizarse únicamente cuando las propiedades numéricas de la aplicación lo permitan.

Inspección del código generado

Una herramienta importante para comprender las optimizaciones consiste en observar el código ensamblador generado por el compilador.

GCC puede generar ensamblador mediante:

```
gcc -O0 -S program.c -o program_00.s
gcc -O3 -S program.c -o program_03.s
```

Posteriormente pueden compararse ambos archivos para observar qué instrucciones fueron eliminadas, reorganizadas o sustituidas. También puede utilizarse:

```
objdump -d program
```

para desensamblar un ejecutable ya construido.

Esta comparación permite relacionar directamente conceptos de programación con la arquitectura del procesador.

Uso de ensamblador dentro de C

Los compiladores como GCC permiten insertar instrucciones de ensamblador directamente dentro de un programa escrito en C. Esta técnica se conoce como **ensamblador embebido** o *inline assembly*. Sin embargo, el uso de ensamblador embebido debe realizarse cuidadosamente. El código generado depende directamente de la arquitectura y puede reducir considerablemente la portabilidad del programa.

Consideración

Este tipo de optimizaciones no son portables.

El ensamblador embebido resulta más interesante cuando se desea acceder a una instrucción especializada de la arquitectura, lo que provee una mejor aceleración en los algoritmos. Esto es ampliamente usado en bibliotecas de procesamiento de video como FFmpeg.

Para generar un código con C incrustado, se usa:

```
asm volatile (
    "instrucciones"
    : operandos_salida
    : operandos_entrada
    : registros_modificados
);
```

Los cuatro componentes principales son:

- **Instrucciones:** código ensamblador que se desea ejecutar.
- **Operandos de salida:** valores producidos por el ensamblador.
- **Operandos de entrada:** valores proporcionados desde C.
- **Clobbers:** registros o recursos modificados por el bloque.

Un ejemplo concreto sobre el uso del ensamblador incrustado es el uso de instrucciones especiales en las microarquitecturas.

Considérese el problema de determinar cuántos bits poseen el valor lógico uno dentro de un entero de 64 bits. Por ejemplo:

0x000000000000000F

posee cuatro bits activos, por lo que:

$$\text{popcount}(0xF) = 4. \quad (2.2)$$

Una implementación sencilla en C puede recorrer individualmente todos los bits:

```
#include <stdint.h>

unsigned int popcount_c(uint64_t value) {
    unsigned int count = 0;

    for (int i = 0; i < 64; ++i) {
        count += value & 1;
        value >>= 1;
    }

    return count;
}
```

Esta implementación realiza aproximadamente 64 iteraciones para cada valor procesado, involucrando múltiples instrucciones, como operaciones lógicas, acumulaciones, bifurcaciones, entre otras.

Algunos procesadores x86 incorporan la instrucción:

POPCNT

la cual calcula directamente la cantidad de bits activos de un registro.

Utilizando ensamblador embebido de GCC puede escribirse:

```
#include <stdint.h>

unsigned int popcount_asm(uint64_t value) {
    uint64_t result;

    asm (
        "popcnt %1, %0"
        : "=r"(result)
        : "r"(value)
    );

    return (unsigned int) result;
}
```

La expresión:

```
"=r"(result)
```

indica al compilador que el resultado debe almacenarse en un registro de propósito general.

Mientras que:

```
"r"(value)
```

indica que el operando de entrada debe encontrarse en un registro.

El compilador puede generar conceptualmente una secuencia cercana a:

```
popcnt %rax, %rax
```

en lugar de ejecutar un ciclo de 64 iteraciones. Esto le permite hacer la ejecución con una única instrucción en lugar de usar 64 paquetes de instrucciones.

La aceleración obtenida depende de la microarquitectura. Sin embargo, este ejemplo muestra una idea fundamental: una operación compleja implementada mediante múltiples instrucciones generales puede ser reemplazada por una instrucción especializada soportada directamente por hardware.

Alineamiento y programación orientada en caché

Vectorización manual

Compilar no es optimizar

Una consideración fundamental es que activar `-O3` no constituye por sí solo una metodología de optimización. El rendimiento de un programa depende de múltiples factores, entre ellos:

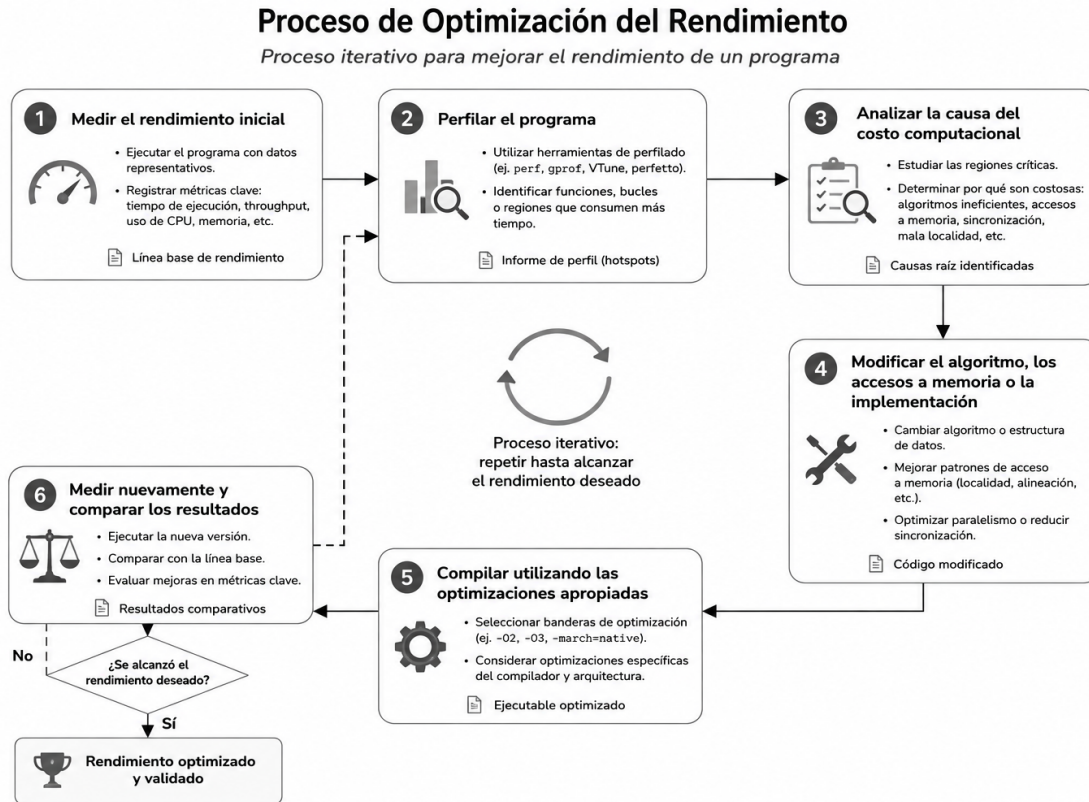


Figura 2.13: Proceso de optimización de un programa

- Complejidad del algoritmo.
- Cantidad de instrucciones ejecutadas.
- Localidad temporal y espacial.
- Comportamiento de las memorias caché.
- Cantidad de accesos a la memoria.
- Predicción de bifurcaciones.
- Paralelismo disponible.
- Utilización de instrucciones SIMD.
- Características de la microarquitectura.

Por esta razón, una metodología básica de optimización puede seguir el proceso descrito en la Figura 2.13, que inicia con el código inicial, medición del rendimiento, el perfilado, análisis de costo

computacional, optimizaciones y reiteración.

Consideración

Comenzar un proyecto con código optimizado no es una buena práctica. A este anti-patrón se le llama *optimización prematura* e implica gastar más tiempo del necesario para tener un mínimo funcional.

Herramientas como `perf`, los reportes de vectorización de GCC y el desensamblado permiten determinar si una modificación realmente mejora la ejecución.